

HIGH-PERFORMANCE & PARALLEL COMPUTING

Shared-Memory Concurrency & OpenMP Paradigms

Laboratory Report - Labs 1-5 + Lab 6 Bonus Challenge

Student: Mardan Ibragimov

Student ID: 230103054

Course: Parallel Programming

Instructor: Sufyan bin Uzayr

Environment: Windows 11 Pro, C++17, OpenMP

Date: 25 September 2026

All benchmark values in this report come from the student's executed programs and generated CSV files.

Contents

1. System & Hardware Specifications
2. Experimental Methodology
3. Empirical Results & Visualizations
 - 3.1 Lab 1 - Fork-Join Model and Oversubscription
 - 3.2 Lab 2 - Numerical Integration and Reductions
 - 3.3 Lab 3 - Work-Sharing and Scheduling
 - 3.4 Lab 4 - Cache Coherency and False Sharing
 - 3.5 Lab 5 - Recursive Task-Based Merge Sort
 - 3.6 Lab 6 Bonus - Pipeline Parallelism
4. Analytical & Discussion Responses
5. Conclusions & Insights
- Appendix A - Execution Evidence
- Appendix B - Reproduction Commands and Deliverables

Executive Summary

This report evaluates shared-memory parallel programming behavior using C++17 and OpenMP across thread-team creation, numerical reductions, loop scheduling, cache-line false sharing, recursive task parallelism, and a three-stage producer-compute-consumer pipeline. The experiments were executed on a Windows laptop with 20 logical processors exposed to OpenMP. Raw measurements were written to CSV files by each program and are included in the repository for reproducibility.

The strongest numerical scaling result was obtained in Lab 2, where the OpenMP reduction reached a 7.21x speedup with 16 threads. Lab 4 showed a 2.85x improvement from 64-byte cache-line padding at 16 threads. Lab 5 demonstrated task-granularity collapse clearly: cutoff $K=1$ took 9.564 s, while practical cutoffs near 1,000-100,000 completed in about 0.10 s. The Lab 6 bonus pipeline became increasingly compute-bound as filter work increased: throughput fell from 1121.5 to 401.3 packets/s while average latency rose from 11.1 to 23.9 ms.

1. System & Hardware Specifications

The hardware identity is consistent with the same Windows laptop used in the student's earlier parallel-computing work. OpenMP independently reported 20 available processors during Lab 1, matching the processor specification below.

Item	Specification
CPU	12th Gen Intel Core i7-12700H
Physical cores	14 total (6 Performance + 8 Efficient cores)
Logical processors / threads	20
L1 cache	~1.22 MiB aggregate instruction + data cache (architecture-derived)
L2 cache	11.5 MB total
L3 cache	24 MB Intel Smart Cache
Operating system	Windows 11 Pro
Compiler	GCC 13.2.0 (MSYS2 Rev3), C++17
OpenMP runtime	GNU libgomp, enabled with -fopenmp
Timing API	omp_get_wtime()

Note: Intel documents 14 cores, 20 threads and 24 MB Smart Cache for the i7-12700H. The executable metadata confirms GCC 13.2.0 from the MSYS2 project. The aggregate L1 value is derived from the Golden Cove and Gracemont per-core cache organization rather than from a Windows counter captured during this run.

2. Experimental Methodology

All programs were compiled with optimization enabled using the same command pattern: `g++ -O2 -std=c++17 -fopenmp <source>.cpp -o <program>.exe`. Dynamic OpenMP team resizing was disabled where appropriate with `omp_set_dynamic(0)`. Timings were collected with `omp_get_wtime()`, and benchmark programs generated CSV files directly so that plots and tables could be recreated without manual transcription.

Experiment	Parameters
Lab 1	10 fork-join runs; $P=\{1,2,4,8,16,32,64\}$; 20 timing repetitions for team create/join; CPU saturation with 10,000,000 sqrt operations per thread.
Lab 2	Race test $N=10,000,000$; critical-section test $N=1,000,000$; reduction scaling $N=100,000,000$; $P=\{1,2,4,8,16\}$; 5 trials per P .
Lab 3	Mandelbrot 1920×1080 , $\text{MAX_ITER}=1000$; static/dynamic/guided; $P=\{2,4,8,16\}$; $\text{chunk}=\{1,16,64,256\}$; 3 trials per cell; work imbalance measured using total escape iterations per thread.
Lab 4	100,000,000 counter updates per thread; unpadding, 64-

	byte padded, and thread-local variants; $P=\{1,2,4,8,16\}$.
Lab 5	5,000,000 random integers; recursive OpenMP tasks; cutoff $K=\{1,10,100,1000,10000,50000,100000\}$; assertion-based correctness check.
Lab 6 Bonus	120 packets of 200,000 floats; bounded queues of capacity 8; filter passes= $\{1,3,6,12\}$; throughput, latency, and maximum queue occupancy recorded.

Measurement limitations: Lab 1 team-creation regions are so short that several values fall below the effective timing granularity of the Windows/OpenMP runtime and appear as zero. They are treated as "below measurable resolution," not as zero-cost thread creation. The provided screenshots do not contain a Task Manager CPU utilization trace, so no numerical CPU-utilization percentage is claimed in this report.

3. Empirical Results & Visualizations

3.1 Lab 1 - Fork-Join Model, Team Creation, and Thread Scoping

Ten consecutive four-thread executions produced different print orders, confirming non-deterministic scheduling. Example orders saved by the program include 3-1-2-0, 2-1-3-0, 2-3-1-0, and 1-3-0-2. The master thread is not guaranteed to print first because the output itself occurs inside a parallel region.

Threads P	Mean create + join time (ms)
1	0.0000
2	0.0500
4	0.0000
8	0.1000
16	0.1500
32	0.2500
64	0.4000

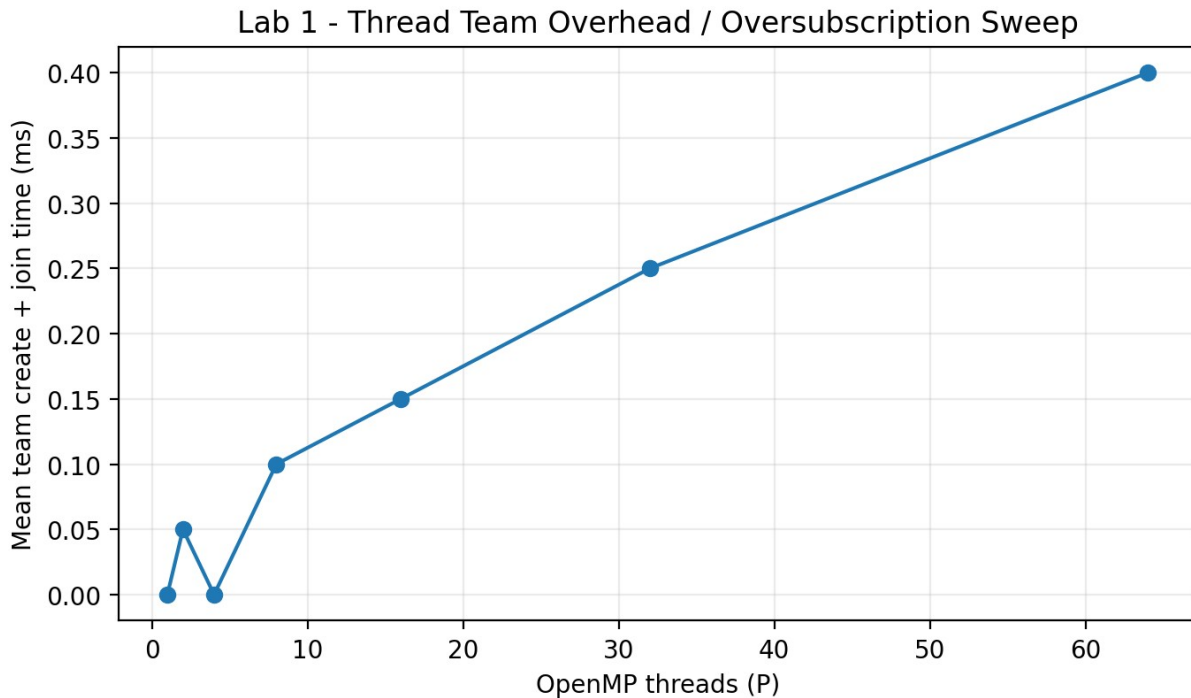


Figure 1. Lab 1 thread-team creation/join overhead. Zero-valued points are below effective timer resolution.

The larger teams show the expected overhead trend: 32 and 64 threads required more creation/join time than smaller teams. Since the machine exposes 20 logical processors, P=32 and P=64 are oversubscribed and necessarily require software threads to share hardware execution resources.

3.2 Lab 2 - Numerical Integration (Pi) & Parallel Reductions

Threads	Calculated pi	Abs. error	Time (s)
1	3.141592653590	6.21725e-14	0.0130

2	1.287002217587	1.85459	0.0090
4	0.979914652507	2.16168	0.0030
8	0.721440683678	2.42015	0.0040

With one thread the naive method remains numerically correct (error $6.22\text{e-}14$), but the shared unsynchronized accumulator collapses as P increases: at 8 threads the computed value is 0.721441, an absolute error of 2.420. This is a direct empirical signature of lost updates.

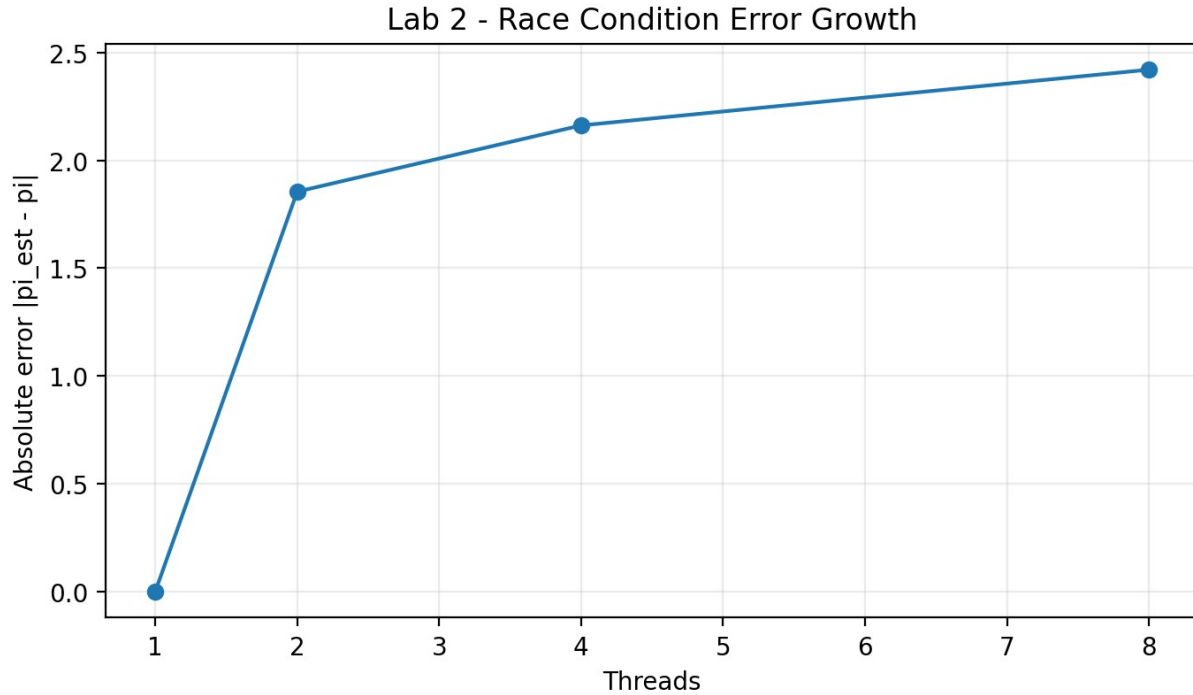


Figure 2. Absolute error of the intentionally racy shared accumulator.

The critical-section variant restored correctness but increased execution time from 0.0020 s to 0.0480 s, corresponding to approximately 2300.2% overhead relative to the one-thread baseline. Every iteration serializes on the same lock, so contention dominates the small arithmetic kernel.

P	Average T(P) (s)	Speedup	Efficiency
1	0.1298	1.000x	100.0%
2	0.0648	2.003x	100.2%
4	0.0334	3.886x	97.2%
8	0.0254	5.110x	63.9%
16	0.0180	7.211x	45.1%

Lab 2 - Strong Scaling: Measured vs Ideal

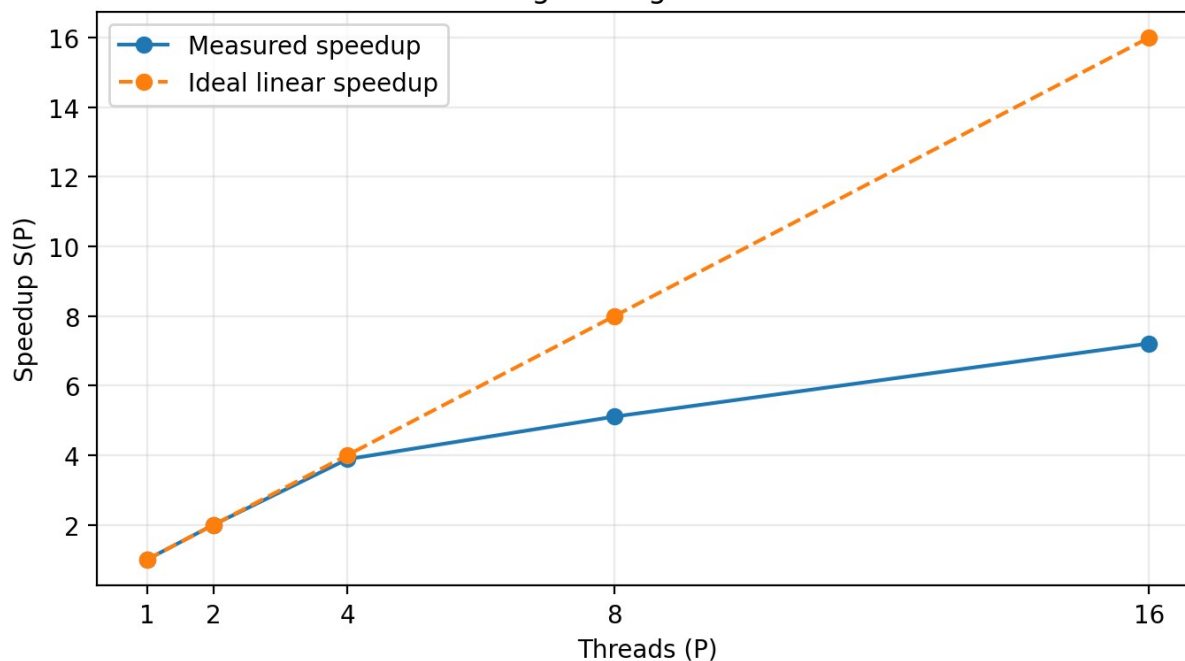


Figure 3. Measured OpenMP reduction speedup compared with ideal linear scaling.

Lab 2 - Parallel Efficiency

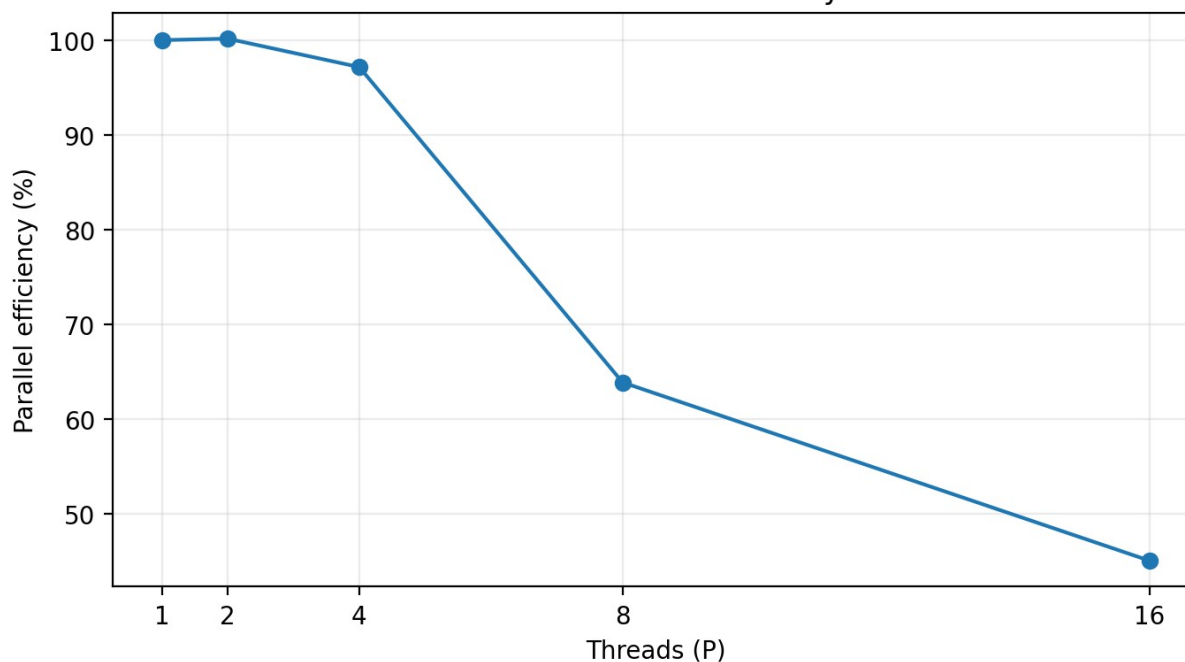


Figure 4. Parallel efficiency of the reduction benchmark.

The best measured speedup was 7.21x at 16 threads. Efficiency decreased to 45.1% at 16 threads, indicating growing synchronization, scheduling, memory-system, and heterogeneous-core effects. The 2-thread efficiency slightly above 100% is measurement noise / favorable cache or turbo behavior rather than super-linear theoretical scaling.

3.3 Lab 3 - Work-Sharing & Loop Scheduling Policies (Mandelbrot)

Each configuration was executed three times and averaged. The checksum was constant at 203,116,879 for all configurations, providing a correctness guard: scheduling policy changes performance and work distribution, but not the rendered numerical result.

Static P\C	1	16	64	256
2	0.3147	0.3153	0.3290	0.3817
4	0.1563	0.1593	0.1700	0.3837
8	0.0930	0.0897	0.2933	0.8097
16	0.1273	0.1247	0.2937	0.6690

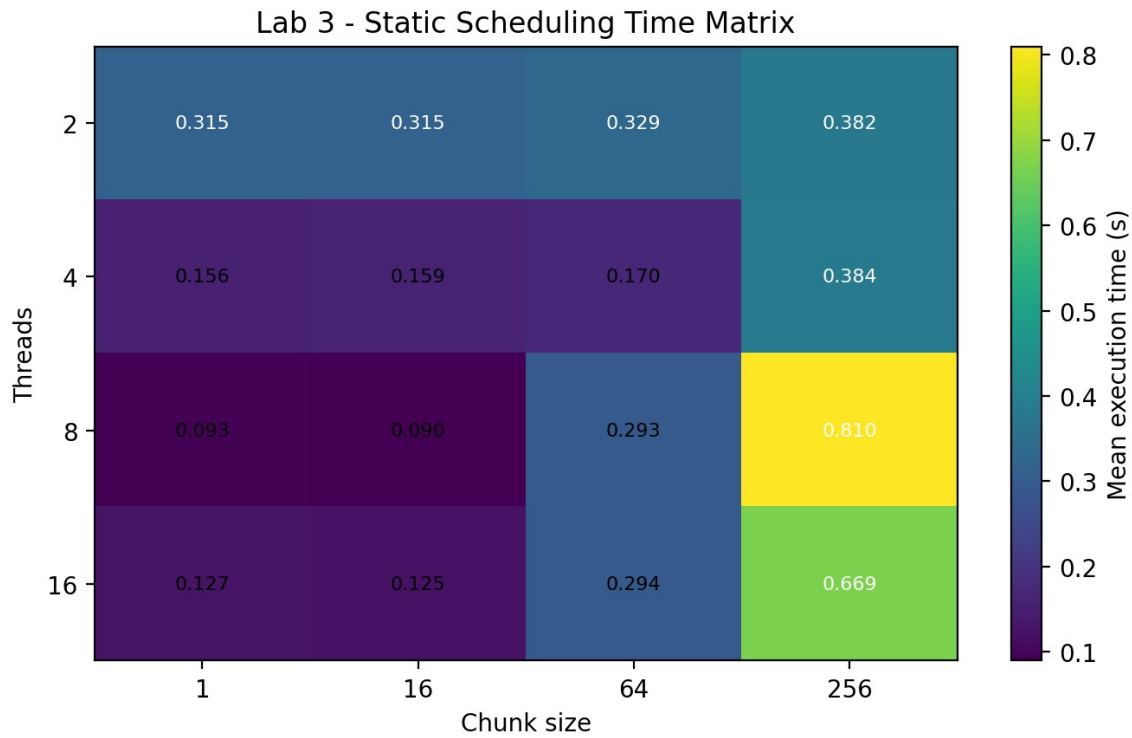


Figure 5. Static scheduling mean execution time (seconds).

Dynamic P\C	1	16	64	256
2	0.3140	0.3130	0.3263	0.3813
4	0.1590	0.1557	0.1687	0.3790
8	0.0850	0.0880	0.1560	0.3777
16	0.0503	0.0510	0.1583	0.3810

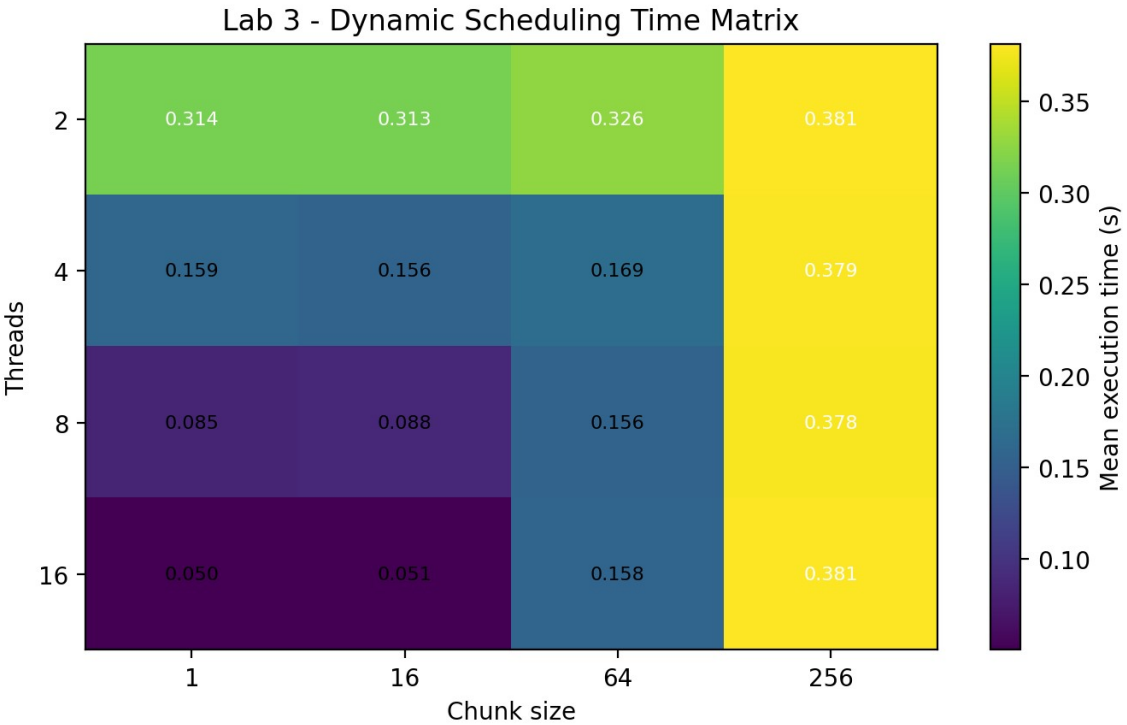


Figure 6. Dynamic scheduling mean execution time (seconds).

Guided P\C	1	16	64	256
2	0.3117	0.3117	0.3123	0.3130
4	0.3380	0.3380	0.3387	0.3473
8	0.1663	0.1640	0.1640	0.3807
16	0.1047	0.0990	0.1557	0.3787

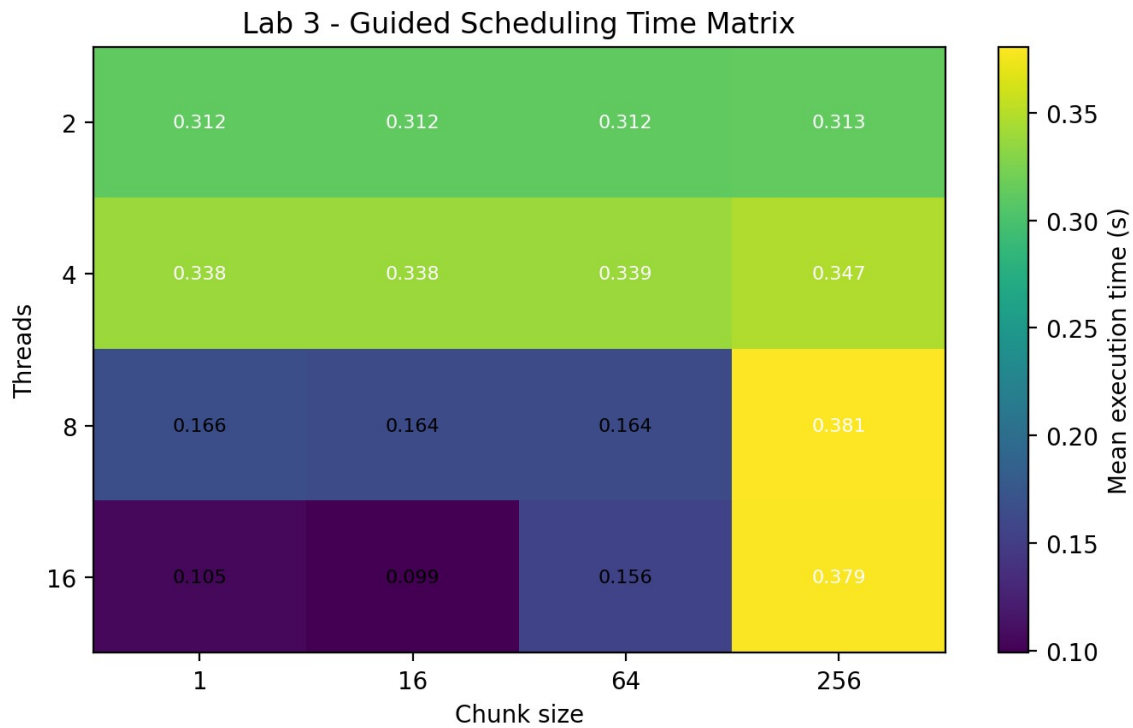


Figure 7. Guided scheduling mean execution time (seconds).

Policy	Best P	Best chunk	Best mean (s)	Measured imbalance
Static	8	16	0.0897	0.161
Dynamic	16	1	0.0503	0.465
Guided	16	16	0.0990	2.327

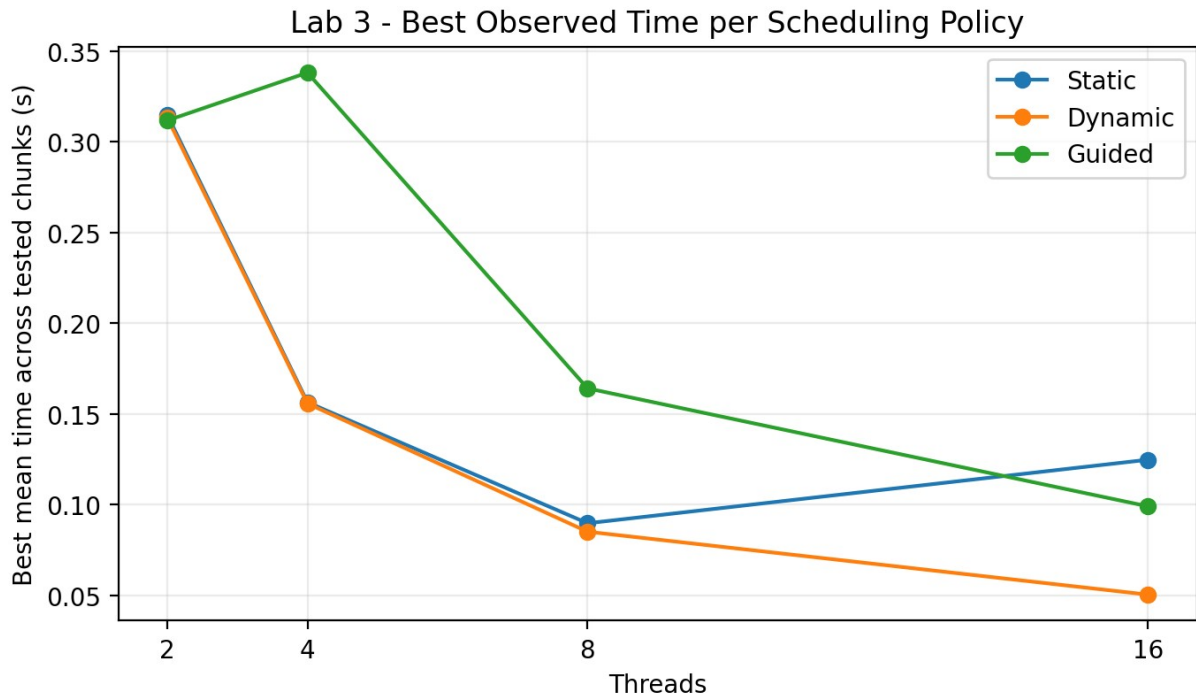


Figure 8. Best observed time per policy for each thread count (minimum across tested chunk sizes).

Dynamic scheduling achieved the fastest configuration in this run: $P=16$ and $\text{chunk}=1$ at 0.0503 s ($\text{chunk}=16$ was nearly identical at 0.0510 s). Static scheduling performed well with fine chunks but degraded strongly for coarse chunks: at $P=16$, increasing chunk size from 1 to 256 increased mean time from 0.1273 s to 0.6690 s and the imbalance metric from 0.018 to 9.656. This illustrates how coarse scheduling can leave some threads with disproportionately expensive Mandelbrot rows.

3.4 Lab 4 - Memory Hierarchy, Cache Coherency, and False Sharing

P	Unpadded (s)	Padded (s)	Thread-local (s)	Padding speedup
1	0.0330	0.0330	0.0310	1.00x
2	0.0380	0.0330	0.0310	1.15x
4	0.0370	0.0320	0.0370	1.16x
8	0.0770	0.0420	0.0430	1.83x
16	0.1340	0.0470	0.0580	2.85x

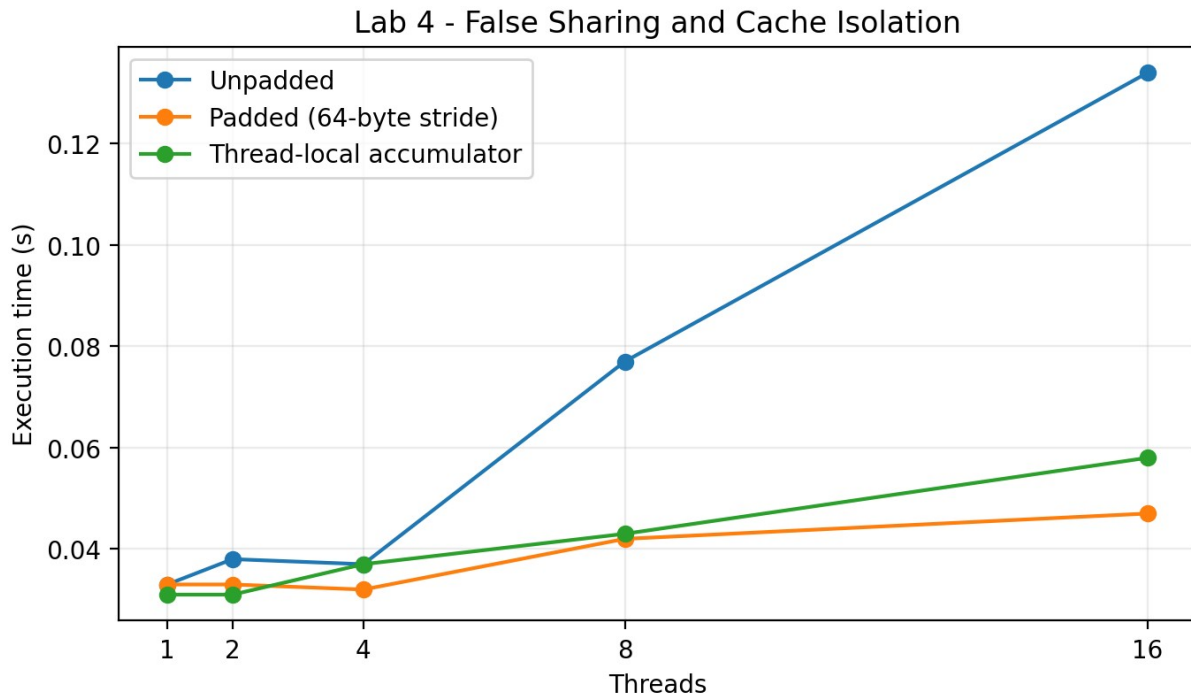


Figure 9. False-sharing scaling: contiguous counters vs 64-byte padding vs thread-local accumulation.

At $P=16$, the unpadded layout required 0.134 s, while 64-byte padding reduced the time to 0.047 s, a 2.85x improvement. This is consistent with cache-line bouncing: independent counters stored on the same 64-byte line repeatedly invalidate one another when written by different cores. The thread-local variant also avoids repeated shared writes and remains close to the padded variant.

Task 4.4 in the manual suggests Linux perf counters. Because this experiment was executed on Windows, Linux-specific L1 miss counters were not collected. The timing curves still provide direct evidence of the false-sharing penalty and its removal through data-layout changes.

3.5 Lab 5 - Recursive Task-Based Parallelism (Parallel Merge Sort)

Cutoff K	Execution time (s)
1	9.5640
10	1.2070
100	0.2540
1000	0.1060
10000	0.1080
50000	0.1040
100000	0.1040

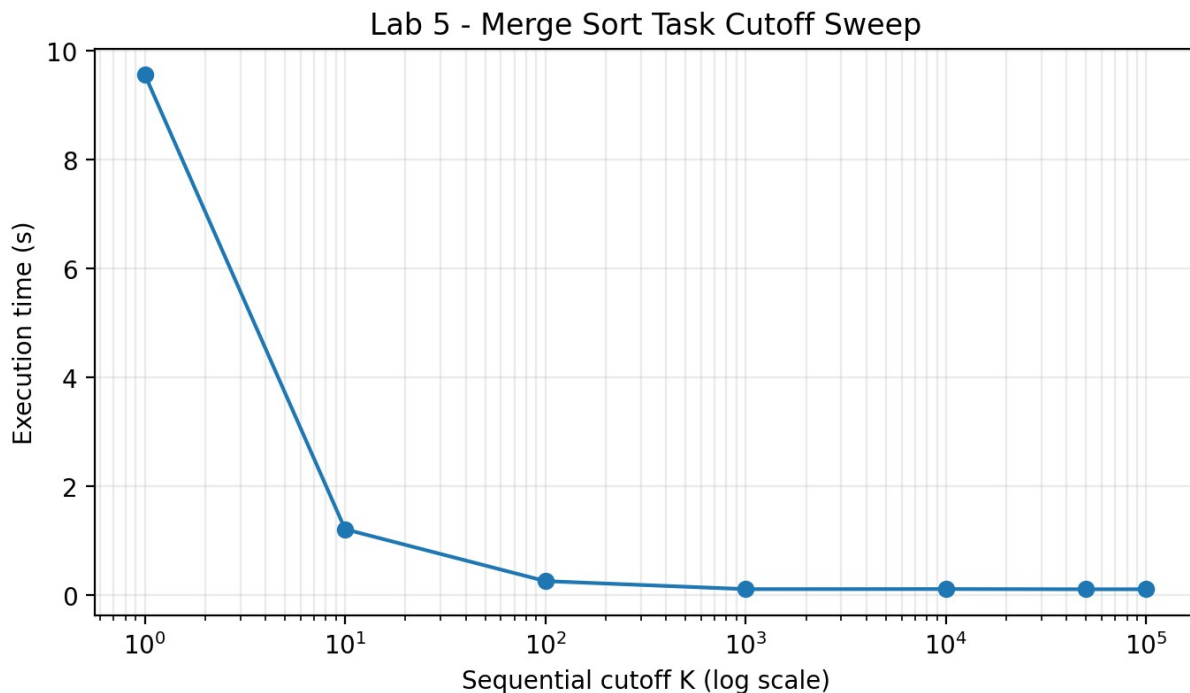


Figure 10. OpenMP task cutoff sweep for 5,000,000 integers.

The output verification passed using `std::is_sorted/assert`. $K=1$ was catastrophic: 9.564 s, because the recursion generates a huge number of tiny OpenMP tasks. Performance improved rapidly as task granularity increased. The best observed time was 0.104 s at $K=50,000$; $K=1,000$ to 100,000 all clustered near 0.10 s, showing a broad practical optimum on this machine.

Empirical one-thread work time T_1 was 0.481 s. With 20 OpenMP threads the measured time was 0.169 s, giving $T_1/T(P) = 2.846$. This empirical ratio is not the literal infinite-processor span; the theoretical Work-Span model is derived in Section 4.5.

3.6 Lab 6 Bonus - 3-Stage Pipeline Parallelism

Filter passes	Total (s)	Throughput (pkt/s)	Avg latency (ms)	Q1 max	Q2 max
1	0.107	1121.50	11.11	8	8
3	0.135	888.89	11.91	8	8

6	0.204	588.24	16.37	8	2
12	0.299	401.34	23.92	8	4

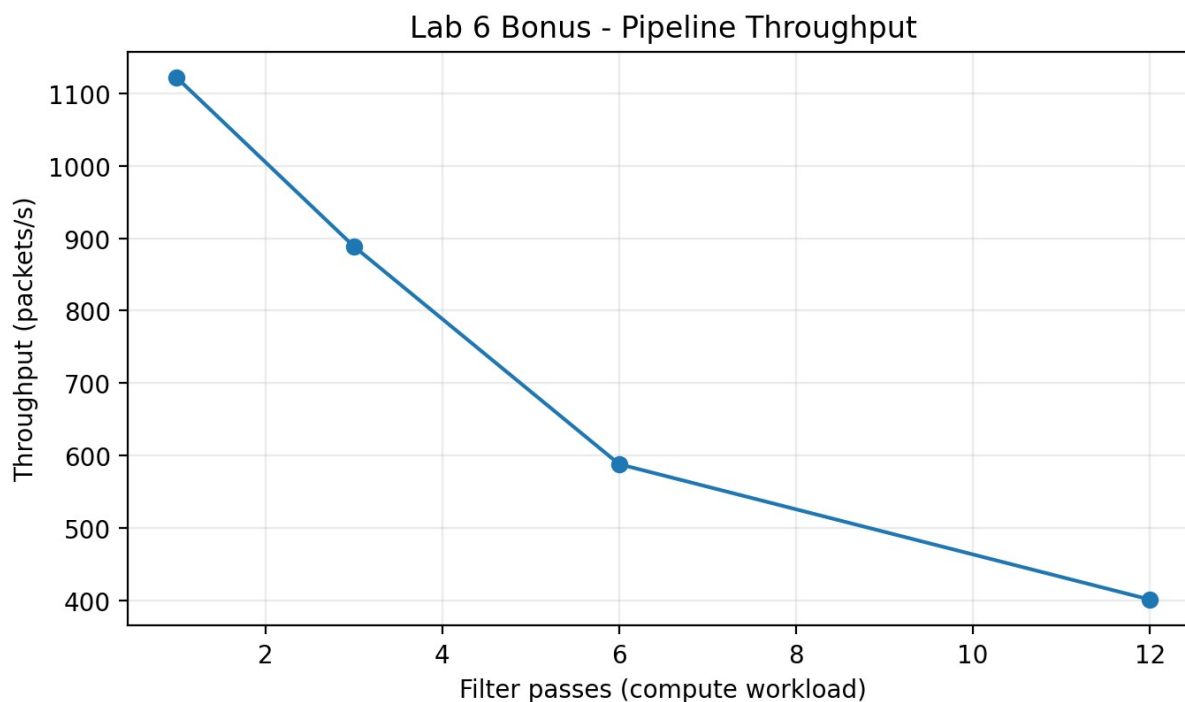


Figure 11. Pipeline throughput as Stage 2 filter workload increases.

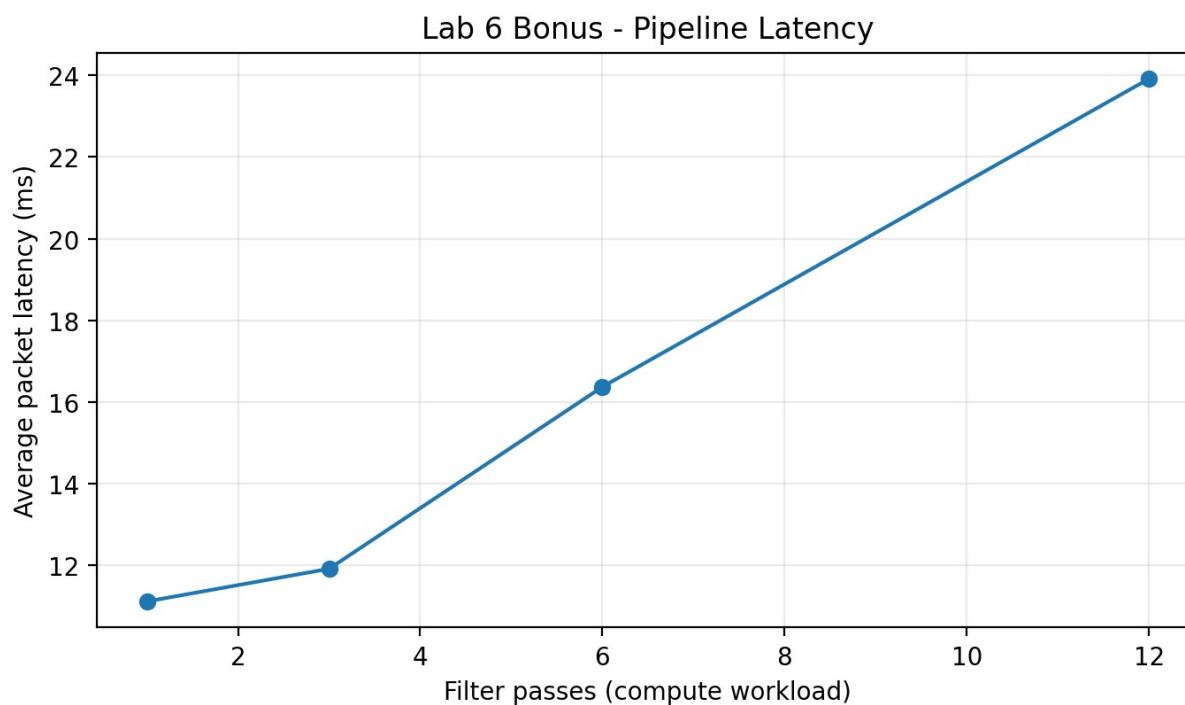


Figure 12. Average end-to-end packet latency as Stage 2 filter workload increases.

The bounded producer-to-compute queue (Q1) reached its full capacity of 8 packets for every tested workload. Meanwhile throughput fell from 1121.5 packets/s at one filter pass to 401.3 packets/s at

twelve passes, and average latency increased from 11.1 ms to 23.9 ms. The combination of a full upstream queue and workload-sensitive throughput identifies Stage 2 as the dominant compute bottleneck for the heavier cases.

4. Analytical & Discussion Responses

4.1 Lab 1 Analytical Questions

Q1.1 Scheduling. Thread IDs print in a different order because OpenMP workers become runnable at nearly the same time, but the operating-system scheduler decides when and on which logical processor each kernel thread executes. Run-queue state, interrupts, background processes, core availability, CPU frequency, and cache state vary from run to run. OpenMP assigns stable logical thread IDs inside the team, but it does not define an output order unless explicit synchronization is added.

Q1.2 Oversubscription. When P exceeds available hardware execution contexts, multiple software threads must time-share the same logical CPUs. Context switches save and restore architectural state such as registers, instruction pointer and stack state. They also disturb caches and TLBs; competing working sets can evict one another, producing cache thrashing. Therefore adding threads beyond the 20 logical processors can increase overhead without adding execution capacity.

Q1.3 Barriers. The implicit barrier at the end of an OpenMP parallel region guarantees that all worker threads have completed the region before serial execution continues. Without that rendezvous, later code could read arrays or partial results while some workers were still updating them, causing races, incomplete reductions, lifetime errors, or logically inconsistent program state.

Q1.4 Architectural Mapping. A hardware execution thread (SMT/Hyper-Threading context) is a set of architectural state inside a physical core and shares much of that core's execution hardware. An OS kernel thread is a schedulable software entity managed by the operating system and can move among hardware contexts. A green or virtual thread is primarily managed by a language/runtime and may be multiplexed onto a smaller number of kernel threads, making creation and blocking cheaper but adding another scheduling layer.

4.2 Lab 2 Analytical Questions

Q2.1 Memory Incoherency / Lost Updates. A normal sum $+=$ term is a read-modify-write sequence: LOAD the old memory value into a register, ADD the new term, then STORE the result. If two cores load the same old sum before either store becomes visible, both compute from that same value and the later store overwrites the earlier update. The arithmetic itself is correct; the interleaving is not atomic. This is why the measured pi value became increasingly wrong with more racing threads.

Q2.2 Reduction Trees. A centralized critical section forces all P threads through one serialization point, so contention grows with the number of participants. A reduction gives each thread a private accumulator and combines only partial sums. A balanced binary merge requires roughly $\log_2(P)$ combine levels, so the synchronization depth grows logarithmically rather than making every loop iteration contend on one global lock.

Q2.3 Amdahl's Law. For serial fraction $f=0.05$, $S(P)=1/(f+(1-f)/P)$. As P approaches infinity, $S_{max}=1/f=20x$. Real scaling can flatten earlier because of OpenMP scheduling overhead, reduction costs, cache capacity and coherence effects, memory bandwidth, load imbalance, heterogeneous P/E cores, finite problem size, and operating-system activity.

Q2.4 Hardware Atomic Primitives. Modern x86 processors normally implement atomic read-modify-write operations by acquiring exclusive ownership of the target cache line through the cache-coherence protocol and preventing competing cores from modifying that line until the locked operation completes. The old idea of locking the external memory bus is not required for ordinary cacheable memory. Architectures such as ARM also use load-exclusive/store-exclusive (LL/SC-style) primitives, where the store succeeds only if no conflicting write has invalidated the reservation.

4.3 Lab 3 Analytical Questions

Q3.1 Fine-Grained Contention. Dynamic scheduling with $chunk=1$ offers excellent theoretical load balance, but every tiny assignment requires scheduler bookkeeping. Threads repeatedly access shared work-distribution metadata, often involving atomics and the same cache lines. The contested resource is therefore the scheduling queue/counter and the cache-coherence/interconnect path protecting it. If each iteration is too cheap, this overhead can exceed useful computation.

Q3.2 Spatial Imbalance. With static row partitioning, threads that receive rows near the central portion of the complex plane tend to become stragglers. Many pixels near or inside the Mandelbrot set do not escape quickly and execute close to $MAX_ITER=1000$ iterations, while outer regions often terminate after only a few iterations. Equal row counts are therefore not equal amounts of computation.

Q3.3 Guided Policy. Guided scheduling starts with relatively large chunks, approximately proportional to the remaining work divided by the number of threads. As the remaining iteration count shrinks, chunk sizes decrease until the configured minimum chunk is reached. Large early chunks reduce scheduler traffic; smaller late chunks reduce the chance that one thread is left with a long tail of expensive iterations.

Q3.4 Engineering Rule of Thumb. Use static scheduling when iteration costs are predictable and uniform because it has the lowest distribution overhead. Use dynamic scheduling when iteration costs are highly irregular or unpredictable and enough work exists per chunk to amortize queue overhead. Use guided scheduling when work is irregular but the total iteration space is large: it usually combines low early scheduling overhead with finer balancing near the end.

4.4 Lab 4 Analytical Questions

Q4.1 MESI State Transitions. Assume four independent 64-bit counters occupy one 64-byte cache line. After both Core 0 and Core 1 read the line, it may be Shared (S) in both private caches. When Core 0 writes its counter, it requests ownership: Core 0 transitions the line to Modified (M), while Core 1's copy becomes Invalid (I). When Core 1 later writes a different counter on that same line, it must obtain ownership; Core 0's Modified copy is invalidated/transferred and Core 1 becomes Modified. Alternating writes repeat this M/I ownership transfer even though the program variables are logically independent.

Q4.2 Bus / Interconnect Bouncing. Adding cores increases the rate at which ownership of the shared cache line migrates among cores. The bottleneck is the coherence fabric, cache hierarchy, and ownership/invalidation traffic rather than arithmetic execution. Useful writes become serialized by line ownership. This explains why the unpadded P=16 case was much slower than the padded case.

Q4.3 True vs False Sharing. True sharing occurs when multiple threads access and modify the same logical variable, so communication is inherent to the algorithm. False sharing occurs when threads modify different variables that merely occupy the same cache line. False sharing can often be removed by padding, alignment, data partitioning, or thread-local accumulation without changing program semantics.

Q4.4 Why @Contended Uses Large Padding. A 64-byte line is the normal x86 cache-line size, but isolating a field by only one line can still allow neighboring fields or adjacent-line prefetch behavior to interact with it. JVM @Contended therefore uses a larger padding width (commonly 128 bytes) so a hot field is separated on both sides, while also accounting for object headers and alignment. The intent is robust isolation rather than merely placing the field at a single 64-byte boundary.

4.5 Lab 5 Analytical Questions

Q5.1 Granularity Breakdown. At cutoff $K=1$, nearly every recursive split becomes an OpenMP task. The runtime must allocate task descriptors, maintain dependency/scheduling metadata, manage recursion stack frames, and push/pop tasks from worker dequeues. Millions of tiny tasks perform very little useful work each, so scheduler and allocation overhead dominates. The measured 9.564 s at $K=1$ versus about 0.10 s near the optimum demonstrates this failure mode.

Q5.2 Work-Stealing Mechanics. Each worker typically owns a double-ended queue. Newly created local tasks are pushed and popped from one end, usually LIFO, which improves cache locality for depth-first work. An idle worker becomes a thief and steals an older task from the opposite end of another worker's deque. Using opposite ends reduces contention between owner and thief, and stealing older tasks tends to obtain a larger subtree of work.

Q5.3 Span Bottleneck. Merge sort has total work $T_1 = \Theta(N \log N)$. With a sequential $O(N)$ merge at each recursion level, the critical path satisfies $T_{\infty}(N) = T_{\infty}(N/2) + \Theta(N) = \Theta(N)$. Therefore theoretical parallelism T_1/T_{∞} is $\Theta(\log N)$, not $\Theta(N)$. For $N=5,000,000$, $\log_2(N)$ is about 22.25, so the asymptotic model suggests only logarithmic parallelism before constants and runtime effects. A parallel merge can reduce span by partitioning one sorted range using binary searches in the other range, recursively merging independent partitions, or by using parallel prefix/merge techniques.

Q5.4 Loop vs Tasking. OpenMP for work-sharing is ideal for regular loops with known iteration spaces and straightforward index partitioning. OpenMP task is superior for irregular dynamic computation: recursive divide-and-conquer, tree search, graph traversal, adaptive meshes, branch-and-bound, and DAG-style workflows where work appears during execution and cannot be represented as one simple canonical loop.

4.6 Lab 6 Bonus Discussion

The implementation uses two bounded blocking queues to connect three concurrently running stages: Producer/Reader $\rightarrow$ Compute/Filter $\rightarrow$ Consumer/Statistics. Condition variables block

producers when a queue is full and consumers when it is empty, eliminating busy-waiting. A sentinel packet provides graceful shutdown. The Q1 occupancy reaching its capacity of 8 across all workloads indicates that the producer can supply work faster than the filter stage consumes it. As filter passes increase, throughput falls and latency rises, so Stage 2 is the principal bottleneck in the tested range.

5. Conclusions & Insights

The experiments demonstrate that parallel performance is determined by much more than thread count. Correct synchronization, work granularity, scheduling policy, and data layout are equally important. Lab 2 showed why a reduction is both safer and more scalable than a shared racy accumulator or a per-iteration critical section. Lab 3 showed that scheduling choices interact strongly with workload irregularity and chunk size. Lab 4 isolated a hardware-level cache-coherence bottleneck that exists even without a logical data race. Lab 5 showed that excessive task creation can erase the benefits of parallelism. The bonus pipeline extended the same reasoning to stage-level concurrency and bottleneck analysis.

- Correctness first: unsynchronized shared updates can produce fast but meaningless results.
- Reductions scale better because most arithmetic occurs in private accumulators and only partial results are combined.
- Fine-grained dynamic scheduling improves balance only when scheduler overhead is small relative to useful work.
- False sharing is a data-layout problem: independent variables can still serialize through cache coherence.
- Task-based parallelism requires a cutoff; $K=1$ was roughly two orders of magnitude slower than the practical region in this experiment.
- Pipeline throughput is limited by the slowest stage; increasing Stage 2 work filled the upstream queue and reduced end-to-end throughput.

Overall, the measured results are consistent with the core OpenMP concepts in the laboratory manual: fork-join execution, reduction, scheduling, cache coherency, task granularity, and pipeline bottlenecks. All source code and raw CSV data are retained in the accompanying repository so every figure can be reproduced.

Appendix A - Execution Evidence

The following screenshots were captured from the actual Windows/VS Code terminal runs used to generate the CSV files in this report.

```

Thread 0 of 4

--- Run 10 | team size = 4 ---
Thread 2 of 4
Thread 3 of 4
Thread 1 of 4
Thread 0 of 4

Task 1.2 τA0 Oversubscription sweep
P= 1 | avg creation+join time = 0.0000000 s
P= 2 | avg creation+join time = 0.0005000 s
P= 4 | avg creation+join time = 0.0000000 s
P= 8 | avg creation+join time = 0.0001000 s
P=16 | avg creation+join time = 0.0001500 s
P=32 | avg creation+join time = 0.0002499 s
P=64 | avg creation+join time = 0.0004000 s

Task 1.3 τA0 CPU saturation
Open Task Manager -> Performance -> CPU now.
Running 20 threads, 1000000 sqrt operations per thread...
Saturation workload finished in 0.0670 s

Generated: lab1_nondeterminism.txt, lab1_oversubscription.csv
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab1>

```

A1. Lab 1 oversubscription and CPU-saturation terminal output.

```

PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab2> g++ -O2 -std=c++17 -fopenmp lab2.cpp -o lab2.exe
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab2> .\lab2.exe
LAB 2 τA0 Numerical Integration (Pi) & Parallel Reductions

Task 2.1 τA0 Race Condition Quantification
P=1 | Pi=3.14159265359 | error=6.217248937901e-14 | time=0.0130 s
P=2 | Pi=1.287002217587 | error=1.854590436003e+00 | time=0.0090 s
P=4 | Pi=0.979914652507 | error=2.161678001082e+00 | time=0.0030 s
P=8 | Pi=0.721440683678 | error=2.420151969912e+00 | time=0.0040 s

Task 2.2 τA0 Critical Section Overhead
Serial time = 0.0020 s
Critical time = 0.0480 s
Lock contention overhead = 2300.18%

Task 2.3/2.4 τA0 Strong Scaling
P= 1 | avg=0.1298 s | speedup=1.000 | efficiency=100.000% | Pi=3.141592653590
P= 2 | avg=0.0648 s | speedup=2.003 | efficiency=100.154% | Pi=3.141592653590
P= 4 | avg=0.0334 s | speedup=3.886 | efficiency=97.156% | Pi=3.141592653590
P= 8 | avg=0.0254 s | speedup=5.110 | efficiency=63.878% | Pi=3.141592653590
P=16 | avg=0.0180 s | speedup=7.211 | efficiency=45.069% | Pi=3.141592653590

Generated: lab2_race.csv, lab2_critical.csv, lab2_scaling.csv
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab2>

```

A2. Lab 2 race, critical-section, speedup and efficiency output.

```

27
Problems Output Debug Console Terminal Ports
P=16 C= 16 | mean=0.0510 s | imbalance=0.2611
P=16 C= 64 | mean=0.1583 s | imbalance=3.9513
P=16 C=256 | mean=0.3810 s | imbalance=9.6563

=== guided scheduling ===
P= 2 C= 1 | mean=0.3117 s | imbalance=0.0037
P= 2 C= 16 | mean=0.3117 s | imbalance=0.0042
P= 2 C= 64 | mean=0.3123 s | imbalance=0.0065
P= 2 C=256 | mean=0.3130 s | imbalance=0.0077
P= 4 C= 1 | mean=0.3380 s | imbalance=2.0700
P= 4 C= 16 | mean=0.3380 s | imbalance=2.0577
P= 4 C= 64 | mean=0.3387 s | imbalance=2.0639
P= 4 C=256 | mean=0.3473 s | imbalance=2.2025
P= 8 C= 1 | mean=0.1663 s | imbalance=2.0087
P= 8 C= 16 | mean=0.1640 s | imbalance=2.0220
P= 8 C= 64 | mean=0.1640 s | imbalance=2.0348
P= 8 C=256 | mean=0.3807 s | imbalance=4.8393
P=16 C= 1 | mean=0.1047 s | imbalance=2.2231
P=16 C= 16 | mean=0.0990 s | imbalance=2.3272
P=16 C= 64 | mean=0.1557 s | imbalance=3.9471
P=16 C=256 | mean=0.3787 s | imbalance=9.6563

Generated: lab3_benchmark.csv, lab3_thread_work.csv
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab3>

```

A3. Lab 3 scheduling benchmark completion and generated CSV files.

```

21
22
Problems Output Debug Console Terminal Ports
P= 8 C=256 | mean=0.3807 s | imbalance=4.8393
P=16 C= 1 | mean=0.1047 s | imbalance=2.2231
P=16 C= 16 | mean=0.0990 s | imbalance=2.3272
P=16 C= 64 | mean=0.1557 s | imbalance=3.9471
P=16 C=256 | mean=0.3787 s | imbalance=9.6563

Generated: lab3_benchmark.csv, lab3_thread_work.csv
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab3> cd ..
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054> cd lab4
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab4> g++ -O2 -std=c++17 -fopenmp lab4.cpp -o lab4.exe
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab4> .\lab4.exe
LAB 4 TAO Memory Hierarchy, Cache Coherency, and False Sharing
Iterations per thread: 100000000

P= 1 | unpadded=0.0330 s | padded=0.0330 s | local=0.0310 s | unpadded/padded=1.00x
P= 2 | unpadded=0.0380 s | padded=0.0330 s | local=0.0310 s | unpadded/padded=1.15x
P= 4 | unpadded=0.0370 s | padded=0.0320 s | local=0.0370 s | unpadded/padded=1.16x
P= 8 | unpadded=0.0770 s | padded=0.0420 s | local=0.0430 s | unpadded/padded=1.83x
P=16 | unpadded=0.1340 s | padded=0.0470 s | local=0.0580 s | unpadded/padded=2.85x

Task 4.4 note: the manual's perf command is Linux-specific.
On Windows, submit Task Manager/CPU evidence unless your instructor requires WSL/Linux perf.
Generated: lab4_false_sharing.csv
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab4>

```

A4. Lab 4 false-sharing benchmark output.

```

Problems Output Debug Console Terminal Ports
Generated: lab4_false_sharing.csv
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab4> cd ..
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054> cd lab5
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab5> g++ -O2 -std=c++17 -fopenmp lab5.cpp -o lab5.exe
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab5> .\lab5.exe
LAB 5 τA0 Recursive Task-Based Parallelism (Merge Sort)
N=500000, threads=20
Task 5.1 verification PASSED | cutoff=50000 | time=0.1090 s

Task 5.2 τA0 Cutoff sweep
K= 1 | time=9.5640 s
K= 10 | time=1.2070 s
K= 100 | time=0.2540 s
K= 1000 | time=0.1060 s
K= 10000 | time=0.1080 s
K= 50000 | time=0.1040 s
K=100000 | time=0.1040 s

Task 5.3 empirical measurements
T1 (1 thread) = 0.4810 s
T(P) (20 threads) = 0.1690 s
Measured T1/T(P) = 2.8462
Generated: lab5_cutoff_sweep.csv, lab5_work_span.csv
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab5>

```

A5. Lab 5 cutoff sweep and empirical T1/T(P) output.

```

Problems Output Debug Console Terminal Ports
K= 1 | time=9.5640 s
K= 10 | time=1.2070 s
K= 100 | time=0.2540 s
K= 1000 | time=0.1060 s
K= 10000 | time=0.1080 s
K= 50000 | time=0.1040 s
K=100000 | time=0.1040 s

Task 5.3 empirical measurements
T1 (1 thread) = 0.4810 s
T(P) (20 threads) = 0.1690 s
Measured T1/T(P) = 2.8462
Generated: lab5_cutoff_sweep.csv, lab5_work_span.csv
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab5> cd ..
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054> cd lab6_bonus
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab6_bonus> g++ -O2 -std=c++17 -fopenmp lab6_bonus.cpp -o lab6_bonus.exe
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab6_bonus> .\lab6_bonus.exe
LAB 6 BONUS τA0 3-Stage Pipeline Parallelism
passes= 1 | total=0.107 s | throughput=1121.50 packets/s | avg latency=0.0111 s | q1 max=8 | q2 max=8
passes= 3 | total=0.135 s | throughput=888.89 packets/s | avg latency=0.0119 s | q1 max=8 | q2 max=8
passes= 6 | total=0.204 s | throughput=588.24 packets/s | avg latency=0.0164 s | q1 max=8 | q2 max=2
passes=12 | total=0.299 s | throughput=401.34 packets/s | avg latency=0.0239 s | q1 max=8 | q2 max=4
Generated: lab6_pipeline.csv
PS C:\Users\Mardan Ibragimov\Desktop\openmp_230103054\lab6_bonus>

```

A6. Lab 6 bonus pipeline throughput/latency output.

Appendix B - Reproduction Commands and Deliverables

Compile from each lab directory with:

```
g++ -O2 -std=c++17 -fopenmp lab1.cpp -o lab1.exe
```

```
g++ -O2 -std=c++17 -fopenmp lab2.cpp -o lab2.exe
```

```
g++ -O2 -std=c++17 -fopenmp lab3.cpp -o lab3.exe
```

```
g++ -O2 -std=c++17 -fopenmp lab4.cpp -o lab4.exe
```

```
g++ -O2 -std=c++17 -fopenmp lab5.cpp -o lab5.exe
```

```
g++ -O2 -std=c++17 -fopenmp lab6_bonus.cpp -o lab6_bonus.exe
```

Repository deliverables:

- Formal laboratory report in PDF and DOCX format.
- Commented C++ source code for Labs 1-5 and Lab 6 bonus.
- Raw CSV benchmark data used to generate every result table and graph.
- Lab 1 non-deterministic execution-order text output.
- Execution screenshots for Labs 1-6.
- README with compilation and reproduction instructions.

References

1. Sufyan bin Uzayr, Laboratory Practice Manual: Shared-Memory Concurrency & OpenMP Paradigms (course-provided manual).
2. Intel, Intel Core i7-12700H Processor Specifications: 14 cores, 20 threads, 24 MB Intel Smart Cache, up to 4.70 GHz.
3. OpenMP Architecture Review Board, OpenMP API concepts: parallel regions, loop scheduling, reductions, tasks, and synchronization.
4. Hardware cache-organization references used only to describe Alder Lake P-core/E-core L1 organization; benchmark conclusions themselves are based on the measured CSV data.